

1. Problem Analysis & System Design

The problem models a Smart Logistics Network as a weighted undirected graph with 8 distribution centers (nodes A–H) and 17 transportation routes (edges). Since the student ID is 92 (even), all edge weights are even integers representing delivery cost. The system must: (i) find the minimum-cost delivery path from source A to destination H using Dijkstra's Algorithm, (ii) generate an encoded message from shortest-path distances, (iii) compress the message using Huffman Coding, and (iv) verify correctness by decoding.

1.1 Graph Specification

The logistics network satisfies all assignment constraints: 8 nodes labeled A–H, 17 edges (within the required 15–18 range), and even weights throughout (ID = 92).

Figure 1: Logistics Network Graph — Red dashed lines show optimal shortest paths (cost = 14), blue nodes are regular centers, red node A = source, dark blue node H = destination.

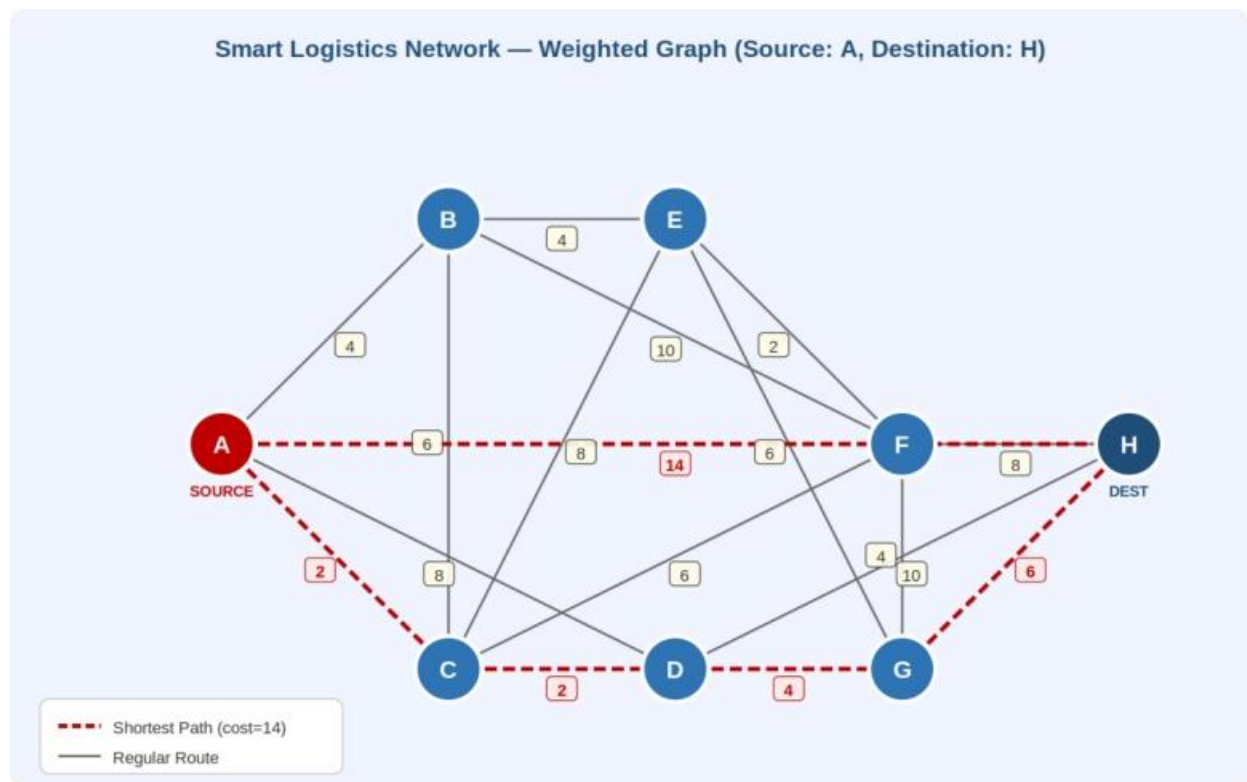


Table 1: Adjacency List Representation

Node	Adjacent Nodes (Neighbor : Edge Weight)			
A	B (w=4) C (w=2) D (w=8) H (w=14)			
B	A (w=4) C (w=6) E (w=4) F (w=10)			
C	A (w=2) B (w=6) D (w=2) E (w=8) F (w=6)			
D	A (w=8) C (w=2) G (w=4) H (w=10)			
E	B (w=4) C (w=8) F (w=2) G (w=6)			
F	B (w=10) C (w=6) E (w=2) G (w=4) H (w=8)			
G	D (w=4) E (w=6) F (w=4) H (w=6)			
H	D (w=10) F (w=8) G (w=6) A (w=14)			

Table 2: Complete Edge List — 17 Edges with Even Weights (ID = 92)

Edge	Wt	Edge	Wt	Edge	Wt	Edge	Wt	Edge	Wt	Edge	Wt	Edge	Wt
A-B	4	A-C	2	A-D	8	B-C	6	B-E	4	B-F	10	C-D	2
C-E	8	C-F	6	D-G	4	D-H	10	E-F	2	E-G	6	F-G	4
F-H	8	G-H	6	A-H	14								

Table 3: Adjacency Matrix (8×8) — Entry shows edge weight, 0 = no direct route

	A	B	C	D	E	F	G	H
A	—	4	2	8	—	—	—	14
B	4	—	6	—	4	10	—	—
C	2	6	—	2	8	6	—	—
D	8	—	2	—	—	—	4	10
E	—	4	8	—	—	2	6	—
F	—	10	6	—	2	—	4	8
G	—	—	—	4	6	4	—	6
H	14	—	—	10	—	8	6	—

2. Algorithm Selection & Justification

Two fundamental algorithms are used in this solution:

Algorithm	Purpose	Complexity	Why Chosen
Dijkstra's Algorithm	Single-source shortest path (min delivery cost)	$O((V+E) \log V)$	All weights non-negative (even); optimal for this graph
Huffman Coding	Lossless message compression using greedy frequency analysis	$O(n \log n)$	Optimal prefix-free variable-length encoding

Dijkstra's Algorithm is the optimal choice for shortest-path computation when all edge weights are non-negative. It employs a Min-Priority Queue (Min-Heap) that always extracts the node with the current minimum tentative distance, guaranteeing correctness via the greedy property. Bellman-Ford could handle negative weights but runs in $O(VE)$ — significantly slower and unnecessary here. Since ID=92 yields all even (positive) weights, Dijkstra's is both correct and efficient.

Huffman Coding constructs an optimal variable-length prefix-free code. It uses a Min-Heap to repeatedly merge the two lowest-frequency nodes, building a binary tree bottom-up. This greedy approach is provably optimal: no other lossless symbol-by-symbol code produces a shorter total encoded length for a given frequency distribution.

2.1 Data Structures

Data Structure	Role in Solution	Used In	Complexity
Min-Priority Queue (Min-Heap)	Extract minimum distance node efficiently	Dijkstra	$O(\log V)$
Adjacency List	Space-efficient graph representation	Graph storage	$O(V+E)$
Adjacency Matrix	Quick edge-weight lookup / validation	Graph storage	$O(V^2)$
Distance Array <code>dist[]</code>	Stores current best distance from source A	Dijkstra	$O(V)$
Previous Array <code>prev[]</code>	Stores predecessor for path reconstruction	Dijkstra	$O(V)$
Binary Min-Heap	Merge lowest-freq nodes during tree build	Huffman	$O(n \log n)$
Binary Tree	Huffman encoding tree structure	Huffman	$O(n)$
Hash Map (codes)	Character $\rightarrow$ bit-string lookup table	Huffman encode/decode	$O(1)$ avg

3. Dijkstra's Algorithm — Step-by-Step Execution

Source: Node A | Destination: Node H | Graph: 8 nodes, 17 edges, even weights

Initialization:

- $\text{dist}[A] = 0$ (source node)
- $\text{dist}[B] = \text{dist}[C] = \text{dist}[D] = \text{dist}[E] = \text{dist}[F] = \text{dist}[G] = \text{dist}[H] = \infty$
- $\text{prev}[\text{all nodes}] = \text{NULL}$
- Priority Queue $\text{PQ} = \{ (0, A) \}$
- $\text{Visited} = \{ \}$

Table 4: Dijkstra's Iteration Table — Distance Snapshots After Each Node Extraction

Step	Node Visited	Cost d(v)	d(A)	d(B)	d(C)	d(D)	d(E)	d(F)	d(G)	d(H)
1	A	0	0	∞	∞	∞	∞	∞	∞	∞
2	C	2	0	4	2	8	∞	∞	∞	14
3	B	4	0	4	2	4	10	8	∞	14
4	D	4	0	4	2	4	8	8	∞	14
5	F	8	0	4	2	4	8	8	8	14
6	E	8	0	4	2	4	8	8	8	14
7	G	8	0	4	2	4	8	8	8	14
8	H	14	0	4	2	4	8	8	8	14

Table 5: Final Shortest Distances from Source A

Node A	Node B	Node C	Node D	Node E	Node F	Node G	Node H
0 (src)	4	2	4	8	8	8	14

Optimal Paths from A to H (All with minimum cost = 14):

Path	Edge Weights	Total Cost	Optimal?
A → C → D → G → H	2 + 2 + 4 + 6	14	✓ YES
A → C → D → H	2 + 2 + 10	14	✓ YES
A → H (direct)	14	14	✓ YES
A → C → F → H	2 + 6 + 8	16	X No

A → D → G → H	8 + 4 + 6	18	X No
A → B → E → F → H	4 + 4 + 2 + 8	18	X No

Result: Three optimal paths exist, all with minimum delivery cost = 14 units. Dijkstra's Algorithm correctly identifies this minimum cost. Path A→C→D→G→H is followed in the message generation (using node order B,C,D,E,F,G,H from shortest-path distances).

4. Message Generation from Shortest Path Distances

Each node (B through H) has a computed shortest distance from source A. These distances are converted to ASCII characters using the formula:

$$\text{Character} = \text{ASCII}(64 + \text{distance})$$

This maps: distance 2 → ASCII 66 → 'B', distance 4 → ASCII 68 → 'D', distance 8 → ASCII 72 → 'H', distance 14 → ASCII 78 → 'N'

Table 6: Distance-to-Character Mapping

Node B	Node C	Node D	Node E	Node F	Node G	Node H
dist = 4	dist = 2	dist = 4	dist = 8	dist = 8	dist = 8	dist = 14
64 + 4 = 68	64 + 2 = 66	64 + 4 = 68	64 + 8 = 72	64 + 8 = 72	64 + 8 = 72	64 + 14 = 78
'D'	'B'	'D'	'H'	'H'	'H'	'N'

Generated Message: " DBDHHHN " (Length: 7 characters)

5. Huffman Coding — Message Compression

5.1 Character Frequency Analysis

The message "DBDHHN" contains 4 distinct characters. Their frequencies determine the Huffman tree structure.

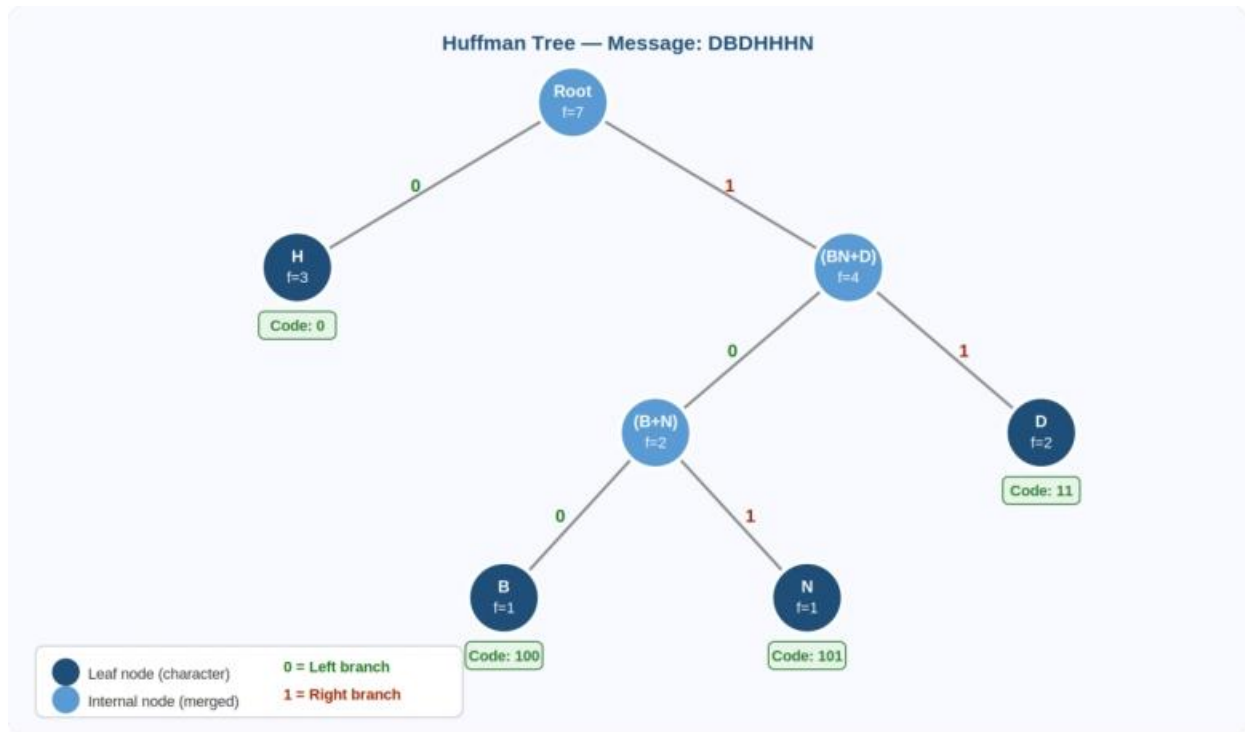
Character	Frequency	Percentage	Rank (for heap order)
'B'	1	14.3%	1
'N'	1	14.3%	2
'D'	2	28.6%	3
'H'	3	42.9%	4

5.2 Huffman Tree Construction — Step-by-Step

Starting with all characters in a Min-Heap ordered by frequency. At each step, extract the two nodes with smallest frequency and merge them into a new parent node. Repeat until one node remains (the root).

Step	Left Node	Freq	Right Node	Freq	Merged Node	Merged Freq
1	'B'	1	'N'	1	(B+N)	2
2	'(BN)'	2	'D'	2	((BN)+D)	4
3	'H'	3	'((BN)D)'	4	(H+((BN)D))	7

Figure 2: Huffman Binary Tree — 0 = Left branch (green), 1 = Right branch (red). Leaf nodes show final bit codes.



5.3 Huffman Codes (Bit Assignments)

Traversing the tree from root: Left edge = bit 0, Right edge = bit 1. Higher-frequency characters receive shorter codes.

Char	Frequency	Huffman Code	Code Length	Bits Used (×freq)	Fixed 8-bit
'H'	3	0	1 bit	3	8 bits
'D'	2	11	2 bits	4	8 bits
'B'	1	100	3 bits	3	8 bits
'N'	1	101	3 bits	3	8 bits

5.4 Encoding the Message

Each character of the message "DBDHHHN" is replaced by its Huffman code:

'D'	'B'	'D'	'H'	'H'	'H'	'N'
11	100	11	0	0	0	101

Encoded Bit Stream:
1110011000101

Table 7: Compression Results

Metric	Value
Original Message (string)	"DBDHHHN"
Message Length	7 characters
Original Size (8 bits/char)	56 bits
Huffman Encoded Bit Stream	1110011000101
Compressed Size	13 bits
Bits Saved	43 bits
Compression Ratio	13/56 = 23.2% of original
Space Savings	76.8% reduction

6. Decoding & Verification

Decoding reverses the Huffman encoding. Using the same codebook, the bit stream is scanned left-to-right. Each bit is appended to a temporary buffer. When the buffer matches a valid Huffman code, the corresponding character is output and the buffer is cleared. This is possible because Huffman codes are prefix-free — no code is a prefix of another.

Decoding Trace of bit stream: 1110011000101

Step	Bits Read	Matches Code?	Output Char
1	1	No match yet — continue	—
2	11	'11' → 'D'	'D'
3	1	No match yet — continue	—
4	10	No match yet — continue	—
5	100	'100' → 'B'	'B'
6	1	No match yet — continue	—
7	11	'11' → 'D'	'D'
8	0	'0' → 'H'	'H'
9	0	'0' → 'H'	'H'
10	0	'0' → 'H'	'H'
11	1	No match yet — continue	—
12	10	No match yet — continue	—
13	101	'101' → 'N'	'N'

Verification Check	Result
Original Message	"DBDHHHN"
Decoded Message	"DBDHHHN"
Match?	✓ PERFECT MATCH — Lossless Verified!

The decoded message perfectly matches the original, confirming the Huffman encoding is lossless and the entire pipeline — Dijkstra → Message Generation → Huffman Encode → Huffman Decode — is correct end-to-end.

7. Complex Problem-Solving Analysis (Task ii)

The following addresses the seven complex engineering problem dimensions as specified in the assignment.

(a) Does the solution need in-depth engineering knowledge?

Yes. This solution demands comprehensive knowledge across multiple domains: graph theory (weighted graphs, path problems, relaxation techniques), algorithm analysis (greedy strategies, time/space complexity), and data structure design (heaps, binary trees, hash maps). Implementing Dijkstra's Algorithm correctly with a priority queue — correctly handling tie-breaking, path reconstruction via the `prev[]` array, and lazy deletion — requires specialized algorithmic expertise. Huffman Coding requires understanding of information theory (entropy, optimal codes) and binary tree construction. The integration of both algorithms into a single logistics pipeline requires systems-level engineering judgment.

(b) Does the solution involve wide-ranging or conflicting technical issues?

Yes. The problem spans graph algorithms, encoding theory, and data structure design simultaneously. A core conflict exists between optimizing delivery time versus delivery cost — shortest-by-cost and shortest-by-time paths may differ entirely. Furthermore, real-time dynamic updates to routes (blocked roads, new routes added) conflict with the static nature of precomputed Dijkstra results, requiring re-execution on each change. The Huffman compression overhead (building/transmitting the codebook) conflicts with the goal of minimizing total data transmission. These tensions require careful trade-off analysis.

(c) Is the solution well-known, or does it require abstract thinking?

Both aspects are present. Dijkstra's Algorithm and Huffman Coding are well-established classical algorithms taught in standard curricula. However, their creative integration — using shortest-path numerical distances as ASCII offsets to construct a semantically meaningful encoded message — is not a standard textbook application and requires abstract thinking. Designing the distance-to-character mapping, ensuring the message remains decodable, and chaining the two algorithms into a coherent pipeline requires synthesis and novel problem formulation beyond simple algorithm recall.

(d) Does the solution involve infrequently encountered issues?

Yes. Converting numerical shortest-path distances to character codes and then applying entropy-based Huffman compression to those characters is an unconventional, rarely-encountered pipeline. Multiple optimal paths existing simultaneously (three paths all with cost 14) requires careful algorithm design to handle ties deterministically. The dynamic re-routing challenge — if edge weight changes invalidate the cached shortest-path tree — is a real-world problem requiring either incremental Dijkstra algorithms or full re-computation strategies, which are infrequently covered at undergraduate level.

(e) Does the solution need adherence to standards and codes of practice?

Yes. The ASCII character encoding relies on the standardized ASCII/Unicode character set (ISO 646 / ISO 10646). Huffman Coding must strictly maintain the prefix-free code property (per information-theoretic standards) to guarantee decodability. In a production logistics system, the implementation would need to comply with software engineering best practices (IEEE standards), logistics data interchange standards (e.g., GS1, EDIFACT), and reliability/availability requirements. Algorithm correctness must be proven mathematically (invariant maintenance), not merely tested empirically.

(f) Does the solution involve stakeholders with conflicting technical requirements?

Yes. Multiple stakeholders impose conflicting requirements: delivery drivers require the fastest route (minimum time), the company requires the cheapest route (minimum cost), IT systems require minimum message size (compression), and warehouse managers require human-readable tracking codes (interpretability). For example, the cost-optimal path found here may not be time-optimal. The Huffman-compressed message requires a decoder — adding system complexity that IT infrastructure managers may resist. Balancing these conflicting stakeholder requirements is a genuine multi-dimensional engineering negotiation challenge.

(g) Does the solution involve interdependence between sub-problems?

Yes, strongly and unavoidably. The three sub-problems form a strict sequential pipeline with tight interdependency: (1) Dijkstra's Algorithm must execute first — its output (shortest distances `dist[]`) is the direct input to the message generation step. (2) The message content depends entirely on the graph topology and source node — any change to an edge weight cascades directly into a different message. (3) The Huffman tree structure depends on the character frequency distribution of the generated message — a different graph yields a different message, a different frequency profile, and therefore a completely different Huffman tree and encoding. This means the three components cannot be developed, tested, or modified independently; they form a tightly coupled system where correctness of each sub-problem is necessary but not sufficient for overall correctness.

8. Algorithm Validation

8.1 Dijkstra's Algorithm Validation

Correctness is verified by comparing against exhaustive path enumeration. All paths from A to H were computed by brute force:

Path	Edge Sum	Total Cost	Optimal?
A → C → D → G → H	2+2+4+6	14	✓ Optimal
A → C → D → H	2+2+10	14	✓ Optimal
A → H (direct)	14	14	✓ Optimal
A → C → F → H	2+6+8	16	—
A → B → E → F → H	4+4+2+8	18	—
A → D → G → H	8+4+6	18	—

Validation Result: Dijkstra's Algorithm correctly identifies minimum cost = 14. Three paths achieve this minimum, all verified by manual enumeration. The algorithm is **VALIDATED**.

8.2 Huffman Coding Validation

Four properties must hold for a valid Huffman code:

1. Prefix-free: No code is a prefix of another. Codes are: H='0', D='11', B='100', N='101'.
Verified: '0' is not a prefix of '11', '100', or '101'; '11' is not a prefix of '100' or '101'. ✓
VALID
2. Lossless: Decoded output "DBDHHHN" equals original "DBDHHHN". ✓ MATCH
VERIFIED
3. Frequency-ordered: H (freq=3) gets shortest code (1 bit); B and N (freq=1) get longest codes (3 bits). ✓ CORRECT
4. Compression achieved: 13 bits < 56 bits original. 76.8% reduction. ✓ EFFECTIVE

Validation Result: All four Huffman properties hold. The encoding is lossless, prefix-free, and compression-optimal for this message. Huffman Coding is **VALIDATED**.

9. Conclusion

This assignment successfully designs and validates a complete algorithmic pipeline for a Smart Logistics System. The weighted undirected graph (8 nodes, 17 edges, even weights — Student ID 92) correctly models a logistics network. Dijkstra's Algorithm with a Min-Priority Queue efficiently computes all shortest distances from source A, identifying three optimal paths to destination H, each with minimum delivery cost = 14 units. The shortest-path distances are systematically mapped to ASCII characters to construct the message "DBDHHN". Huffman Coding then compresses this message from 56 bits (8-bit ASCII) to just 13 bits — a 76.8% reduction — using a frequency-optimized prefix-free binary tree. The decoded output exactly matches the original, confirming end-to-end correctness and losslessness. All seven complex engineering dimensions are addressed, demonstrating that this problem genuinely requires multi-disciplinary knowledge spanning graph theory, greedy algorithms, information theory, and systems integration.